Build Prompt: "KeyForge" Native Piano Technique, Chords and Patterns Trainer (iPhone + iPad)

You are a senior iOS engineer and music-software architect. Build a production-quality **native SwiftUI universal app (iPhone + iPad, iOS/iPadOS 17+)** called **KeyForge**. It teaches adult beginners up to early-intermediate players three things: **(1) finger exercises and technique, (2) chords and scales in all keys, (3) common musical patterns**. The feedback must be honest and come from MIDI. Work in phases, commit often, write tests, and do not stub core logic silently. List every TODO in `TODO.md`.

0. Product principles (non-negotiable)

1. MIDI first. Microphone is a fallback that clearly labels what it cannot judge (dynamics, pedal, dense chords).
2. Honest grading with separate dimensions (pitch, rhythm, evenness, articulation, dynamics, pedal). No blended vanity stars.
3. Real notation by default. Falling-notes view is optional "training wheels" only.
4. Everything is transposable to all 12 keys and generated from parameters, not hand-typed per key.
5. Adult tone: no cartoons, streak freezes, no shaming, explain *why*.
6. Offline first. No account required. iCloud sync optional.
7. Accessibility: VoiceOver, Dynamic Type, color-blind-safe (shape + color), Reduce Motion.
8. **Ethical monetization is a product requirement.** In 2,411 App Store reviews of 17 competitors, billing dark patterns were the #1 complaint: forced plans, auto-renew after cancelling, dozens of upsell pop-ups. Required: trials need no card, cancel or restore takes one tap, the app reminds the user 3 days before renewal, there are no interstitial upsells during practice, and there are no ads.
9. **Reliability is a feature.** Competitors are called "abandonware" over broken MIDI after iOS updates. Required: survive MIDI hot-plug and Bluetooth drop mid-take, a 99.8%+ crash-free-session target, and no network dependence for practice.
10. **Healthy technique.** Follow Hanon-Faber and modern pedagogy: Hanon figures are short, relaxed, hands-separate first, with enforced micro-rests and gesture cues. No endurance challenges, no "play until it hurts" streaks. Include an optional wellness check-in (pain/tension 0–3).
11. **Multi-key from day one (Robert Pace).** Every exercise is transposable, and the daily plan rotates keys through the circle of fifths.

1. Tech stack

- Swift 5.10+/Swift 6 strict concurrency. SwiftUI with the Observation framework (`@Observable`). SwiftData for persistence, with CloudKit sync behind a flag.
- **CoreMIDI** with `MIDIClientCreateWithBlock` and `MIDIInputPortCreateWithProtocol` (MIDI 1.0 UMP). Hot-plug handling through notifications. **Bluetooth LE MIDI** pairing UI via `CABTMIDICentralViewController` (wrapped in `UIViewControllerRepresentable`). USB-C/ Lightning class-compliant devices and Network MIDI sessions. Optionally use the MIDIKit Swift package (https://github.com/orchetect/MIDIKit) for convenience. Keep timestamps in host time (`mach_absolute_time`) and convert accurately.
- **Audio:** AVAudioEngine plus `AVAudioUnitSampler` loading a bundled SF2/SFZ piano (use a permissively licensed piano such as Salamander Grand, CC-BY, with attribution). Also a click/metronome node scheduled sample-accurately with `AVAudioPlayerNode.scheduleBuffer(at:)`. Use `AVAudioSession` category `.playAndRecord` with `.mixWithOthers` and `.allowBluetoothA2DP`, and low IO buffer (`setPreferredIOBufferDuration(0.005)`).
- **Latency and timestamps (critical).** Measured iOS BLE-MIDI latency is about 22.5 ms vs. about 3.5 ms for USB (lower bound: the 11.25 ms BLE connection interval). Score timing from the **MIDI packet timestamps**, never from main-thread arrival time. Keep separate calibrated offsets for USB, BLE, network, and mic, and expose a per-transport jitter readout in Settings.

- **Default sound source for MIDI users = the user's own keyboard.** App sound is off by default when MIDI is connected. Offer "app sound" plus local-control instructions as an option. (Users report that app instrument sounds over MIDI cause clock and latency problems.)
- **MIDI out:** optional routing of playback and metronome to the user's keyboard. Local-control note in Settings.
- **Mic pitch detection:**
- Phase 1: monophonic YIN/MPM on an `installTap` buffer with Accelerate/vDSP, with onset detection by spectral flux.
- Phase 2: polyphonic Core ML model. Convert Spotify Basic Pitch (Apache-2.0, ships a Core ML model: https://github.com/spotify/basic-pitch; Swift reference: https://github.com/zhangyufei49/BasicPitchSwift) and run it on overlapping ~1 s windows with reconciliation of note events across windows (Basic Pitch is not streaming; see issue #108). Evaluate Mobile-AMT-style online models later.
- Use a confidence threshold. Never mark a note wrong on low confidence; mark it "unheard" instead.
- **Notation:** Verovio (LGPL, C++). It ships an official Swift Package (`https://github.com/rism-digital/verovio.git`, iOS 16+, `VerovioToolkit.loadData(musicXML)` / `renderToSVG`; see https://book.verovio.org/installing-or-building-from-sources/swift-and-objective-c.html), rendering MusicXML/MEI to SVG, displayed in a `WKWebView` or rendered to SwiftUI via SVG parse. Use Verovio's element IDs to highlight notes. Wrap it behind a `NotationRenderer` protocol so a native renderer can replace it.
- **Charts:** Swift Charts. **Achievements:** Game Center (optional). **Hand pose (Phase 3):** Vision `VNDetectHumanHandPoseRequest`.
- No third-party analytics. Use the on-device-only `os.Logger`.

2. Architecture (modular SPM packages)

```
KeyForgeApp (app target, iOS universal)
Packages/
  MusicCore      — pitch, interval, key, scale, chord, voicing, fingering, rhythm, transposition (pure Swift,
100% unit tested)
  ExerciseGen    — parameterized generators -> Exercise (notes + fingers + timing + metadata)
  InputKit       — MIDIInputService, MicInputService -> unified AsyncStream<PerformanceEvent>
  SoundKit       — sampler, metronome, MIDI out, transport clock
  ScoringKit     — alignment (expected vs played), per-dimension scores, per-finger analytics, diagnosis rules
  SchedulerKit   — skill graph, mastery rules, FSRS spaced repetition, daily plan builder (interleaving)
  NotationKit    — MusicXML builder from Exercise, Verovio bridge, highlight API
  GuitarBridge   — ASCII-tab parser, tuning model, fret<->pitch, tab -> Exercise
  Persistence    — SwiftData models, migrations, CloudKit flag
  DesignSystem   — colors, typography, KeyboardView, FingerBadge, ScoreChip
```

Use MVVM with `@Observable` view models, dependency injection through an `AppEnvironment` container, and protocols for every service so tests can inject mocks (for example, `MockMIDIInput` replays recorded event files).

3. Core data model

```
struct Pitch: Hashable, Codable { var midi: UInt8 }            // spelling via Key context
enum PitchClass: Int, CaseIterable { case c, cs, d, ds, e, f, fs, g, gs, a, as_, b }
struct SpelledNote { var letter: Letter; var accidental: Int; var octave: Int }
struct Key { var tonic: SpelledNote; var mode: Mode }          // spelling-correct (Eb not D#)
struct ScaleType { id; name; intervals: [Int] }                // major, minors x3, modes, penta, blues
struct ChordQuality { id; symbol; intervals: [Int] }           // triads, 7ths, sus, add, 9ths
struct Voicing { quality; inversion: Int; spread: VoicingStyle } // close, open, shell, rootless A/B
struct Fingering { hand: Hand; fingers: [Int] }                // 1-5
enum Hand { case left, right, both }
struct NoteEvent { pitch; startBeat: Double; durationBeats: Double; finger: Int?; hand: Hand; velocityTarget:
ClosedRange<UInt8>? }
struct Exercise { id; title; category: Category; key: Key; tempoBPM: Int; timeSig; events: [NoteEvent];
chordSymbols: [(beat, String)]; params: ExerciseParams; tags }
enum Category { case technique, scale, arpeggio, chordDrill, progression, accompanimentPattern, riff, song,
reading }
struct PerformanceEvent { kind: noteOn/noteOff/cc(sustain); pitch; velocity; hostTime; source: midi/mic;
confidence: Float }
struct TakeResult { exerciseId; date; tempo; scores: DimensionScores; perFinger: [FingerStat]; mistakes:
[Mistake]; passed: Bool }
struct DimensionScores { pitch, rhythm, evenness, articulation, dynamics, pedal: Double? } // nil = not
measurable (mic)
struct SkillNode { id; category; key; hand; targetTempo; prerequisites: [id]; mastery: MasteryState }
struct ReviewCard { id; skillId; fsrsState: stability, difficulty, due }
struct PracticeSession { date; plannedBlocks; completedTakes; minutes }
```

SwiftData `@Model` classes mirror the persisted ones (TakeResult, SkillNode state, ReviewCard, PracticeSession, UserSettings, UserChordChart, ImportedTab).

4. Content generation (ExerciseGen) with required generators and sample output

Every generator takes `(key, hand, octave, tempo, rhythmVariant, repetitions)` and outputs an `Exercise` with fingerings.

Technique 1. `FiveFingerPattern` – up/down 1-2-3-4-5-4-3-2-1 in any key. Sample C major RH: C4 D4 E4 F4 G4 F4 E4 D4 C4, fingers 1 2 3 4 5 4 3 2 1. LH mirrors an octave down, fingers 5 4 3 2 1 2 3 4 5. 2. `HanonFigure(n)` – Hanon No. 1 figure: ascending, the 8-note cell C E F G A G F E is repeated, starting one step higher each time, for two octaves, then descends using the mirror cell. RH fingers 1 2 3 4 5 4 3 2, LH 5 4 3 2 1 2 3 4. Provide Hanon 1–10 as parameterized cells (Hanon is public domain). Transpose with fingering kept ("Hanon in all keys" mode, flagged advanced). 3. `HeldNoteIndependence` – hold 1 and 5 while 2-3-4 repeat a pattern (score: held notes must not release, CC/noteOff check). 4. `ThumbUnder` – C D E then thumb under to F (1 2 3 1). 5. `Trill(fingerPair)`, `RepeatedNote(fingers 3-2-1)`, `ContraryMotion`, `ParallelOctaves`. 6. `RhythmVariant` for any exercise: straight, dotted (long-short), reverse dotted, triplets, accent on every 3rd note.

Scales and arpeggios 7. `Scale(type, key, octaves 1–4, hands, motion parallel/contrary)` with fingering rules: C-group majors (C G D A E): RH 123 1234, LH 54321 321. Include standard exceptions for F (RH 1234 123), B-flat, E-flat, A-flat, D-flat, G-flat/F-sharp, B, and the harmonic/melodic minors, from a **hand-verified fingering table** in `fingerings.json`. Unit test that each table entry's finger count equals its note count. 8. `Arpeggio(chord, inversion, octaves)` with standard fingerings (RH 1 2 3 5 root position triad over 2 octaves: 1 2 3 1 2 3 5).

Chords 9. `TriadInversionDrill(key list)` – root → 1st → 2nd → root up the keyboard, and the same descending. Sample C: [C4 E4 G4] [E4 G4 C5] [G4 C5 E5] [C5 E5 G5]. Fingers RH 1-3-5, 1-2-5, 1-3-5. Graded on "chord lands within 40 ms spread." 10. `ChordRecallCard` – shows a symbol (for example "F#m / 1st inv / LH"). The user plays it and the app grades pitch-class set, inversion (bass note), hand range, and response time. Feeds FSRS. 11. `VoiceLeading(progression)` – the app picks the nearest inversion for each chord, and the user plays it with minimal motion. 12. `SeventhChords`, `ShellVoicings` (LH 1-7 or 1-3), and `RootlessVoicings` (Phase 2).

Patterns 13. `Progression` library (Roman numerals, rendered in any key): I–V–vi–IV, vi–IV–I–V, I–vi–IV–V, ii–V–I (major), iiø–V7–i (minor), 12-bar blues (I7 IV7 V7), i–♭VII–♭VI–♭VII, Pachelbel I–V–vi–iii–IV–I–IV–V, Andalusian i–♭VII–♭VI–V, I–IV turnarounds. 14. `AccompanimentPattern` = rhythm template + voicing rule, applied to a progression: blockWhole, blockQuarters, brokenTriad, **Alberti** (low-high-mid-high in 8ths: in C, C4 G4 E4 G4), root-5-8, balladArpeggio 1-5-8-10, waltz (root, chord, chord), pop8ths (with syncopation on the "and" of 2), boogie bass (1-3-5-6-♭7-6-5-3), stride (bass on 1/3, chord on 2/4) as Phase 2. 15. `RHMelodyPattern`: chord-tone melodies, passing tones, and pentatonic riffs. 16. **Pattern Mixer** screen: choose progression × LH pattern × RH pattern × key × tempo × groove. The app plays a backing (drum loop plus bass) and grades the user's part. The grading accepts any chord tone on comping beats. 17. **Lead-sheet comping**: the user enters a chord chart (text: `| Dm | C | Dm | C |`) and the app scores chord-tone coverage and rhythm only. 18. Sample riff content: "D-minor arpeggiated riff" (original, non-copyrighted). Do **not** ship copyrighted song arrangements. Allow user-imported tabs and charts stored locally only.

Guitar bridge 19. `TabParser`: parses standard 6-line ASCII tab (e|B|G|D|A|E, digits, multi-digit frets, h/p/b/~ ignored or marked as legato). Tuning is configurable. It outputs pitches with frets; rhythm is unknown, so it defaults to even eighths and the user can edit. Unit test: B-string fret 6 = F4 (MIDI 65), high-e fret 0 = E4 (64), low E fret 0 = E2 (40). 20. Fretboard ↔ keyboard side-by-side view with octave labels (C4 = middle C).

Additional generators (from pedagogy research) 21. `TuneUp(key)` (Pace): a short I–V7–I cadence plus a five-finger pattern that establishes the key before each session. 22. `DozenSet(level)`: groups of 12 short warmups. Introduce only 2–3 new ones per session, and add more after mastery (A Dozen a Day model). 23. `TriadRun(quality, key)`: a triad and its inversions run up and down the whole keyboard (forum and teacher advice). Then all four qualities (maj/min/dim/aug), then the ten 4-note chord types, then open position. 24. `TriadPlusMode`: the LH holds a diatonic triad while the RH plays the matching mode up and down (C maj + C Ionian, D min + D Dorian…), for ear integration. 25. `CircleOfFifthsScales`: scales at metronome 60, 2

octaves, hands together, parallel then contrary motion, walking the circle of fifths. 26. `GuitarVoicingFix(progression)`: shows a naive root-position version next to LH root + RH voice-led inversions (G = B D G, C = C E G, D = A D F♯) plus optional add9s. It teaches why guitar chords "sound off" on piano. 27. `KeyTopographyGroups`: groups triads by black-key pattern (all-white: C F G; black in the middle: D E A; black on the outside: D♭ E♭ A♭; others), so a group can be practiced as one motor family. 28. `SongSectionizer`: any user song or chord chart is split into 2–4 bar sections with per-section mastery. Answers the "doesn't break songs into sections" complaint.

5. Scoring engine (ScoringKit)

- **Clock:** a transport with count-in. Expected onsets come in host time. Calibrate latency in Settings by tapping along to clicks, keeping one offset each for MIDI and mic.
- **Alignment:** dynamic-time-warping or greedy windowed matching of played onsets to expected notes (±150 ms window at beginner level, tightening by level). Unmatched played notes = extra/wrong. Unmatched expected notes = missed.
- **Scores:**
- Pitch = correct matched / expected, with penalties for extras.
- Rhythm = mean |deviation| ms mapped to 0–100 (for example, ≤20 ms = 100, ≥120 ms = 0), with rests respected (a note sounding through a rest is penalized).
- Evenness = coefficient of variation of inter-onset intervals and of velocities.
- Articulation = legato overlap (next on − prev off) vs. target (legato ~0–30 ms overlap, staccato < 50% duration).
- Dynamics = velocity within the target range or following a crescendo shape.
- Pedal = CC64 changes within a window after the chord change (catch "mush" when the pedal is held across chord changes).
- Each dimension is nil if unmeasurable (mic mode).
- **Evenness reference:** report inter-onset-interval SD in ms per hand and direction. Show it against a professional reference of about 8–9 ms for 16th-note scales at 120 BPM (Jabusch et al., MIDI scale analysis). Provide level-appropriate targets, for example beginner ≤ 35 ms, intermediate ≤ 20 ms, advanced ≤ 12 ms. These targets are configurable and come from `scoring_targets.json`.
- **Per-finger stats** (from fingering metadata): mean velocity, velocity SD, mean onset deviation, error count. Render a hand heatmap.
- **Diagnosis rules engine** (data-driven JSON rules), for example: "finger 4 or 5 is late by more than 15 ms against the group average → suggest RhythmVariant dotted drill," and "velocity spikes on thumb crossings → suggest thumb-under slow drill."
- **Modes:** Learn (wait for correct note, no score), Practice (loopable, scored, tempo adjustable), Test (in tempo, no waiting, counts toward mastery).
- **Wait mode must not count toward mastery.**

6. Scheduler (SchedulerKit)

- Skill graph seeded from `curriculum.json` (see §8). A node is mastered when a Test take meets criteria (for example pitch ≥ 95, rhythm ≥ 85, evenness ≥ 80) at the target tempo **on 2 different calendar days**.
- **Tempo ladder:** after 3 clean passes in a row, raise BPM by +4 to +8 (configurable). If a score drops under threshold twice, lower it by 8.
- **Daily plan builder:** input is the minutes available (5/10/20/30). Output is warmup (10%), new-skill block (blocked practice, ~25%), interleaved review (2–4 skills rotating every 2–3 takes, ~40%), and pattern or application (~25%). FSRS (open-source algorithm) handles chord and scale recall cards. Note in-app: "Rest and sleep consolidate motor learning; tomorrow's check matters more than today's."
- Health: suggest a break after 25 minutes of continuous technique work, and limit high-tempo repetition volume.

7. Screens

1. **Onboarding:** experience level, reads notation? (y/n), guitar player? (enables Tab bridge), connect MIDI (BT/USB, test by pressing a key), latency calibration, daily minutes, goal song (optional).
2. **Today:** the generated plan with cards, streak (with freezes), minutes this week, and a "Start" button.

3. **Practice Stage** (core): notation pane plus an 88-key `KeyboardView` (scrolls and zooms to the relevant range; finger numbers on keys; LH blue, RH yellow; wrong notes as a red outline plus an X shape), plus transport (play demo, loop bars A–B, start from bar, tempo slider 30–150%, metronome, count-in, hands L/R/both, "app plays other hand"), plus view toggle Staff / Keys / Falling / Tab. On iPad landscape everything shows at once; on iPhone, the panes are tabbed. After each take: a score sheet with dimension chips, finger heatmap, mistakes jump list, and diagnosis tip.
4. **Technique Gym:** list of generators, and per-exercise personal-best tempo charts.
5. **Chords & Scales:** circle-of-fifths browser, chord/scale detail (staff, keys, fingering both hands, intervals, hear block or arpeggiated), drills, recall deck, and Reverse ID (play any chord → name with alternatives).
6. **Pattern Mixer** and **Progression library**.
7. **Lead Sheet / Tab Import:** paste text, preview, edit rhythm, save.
8. **Progress:** Swift Charts trends (tempo over time, per-dimension scores, minutes, keys mastered grid 12 × skill).
9. **Settings:** MIDI devices, MIDI out/local control, latency, sound, note naming (English/solfège/German), left-handed layout, accessibility, iCloud, export data (JSON/CSV), privacy.
10. **Paywall** (Phase 2): clear free vs. Pro, a lifetime option, no-card trial, restore purchases (StoreKit 2).

8. Curriculum content (`curriculum.json`)

Levels 1–6, each a set of skill nodes referencing generators and parameters. Example excerpt:

```
{"levels":[
 {"id":"L1","title":"Five Fingers & First Triads","nodes":[
  {"id":"tech.5f.C.RH","gen":"FiveFingerPattern","key":"C","hand":"R","targetBPM":80},
  {"id":"tech.5f.C.LH","gen":"FiveFingerPattern","key":"C","hand":"L","targetBPM":80},
  {"id":"chord.triad.C.F.G.root","gen":"ChordRecallCard","keys":["C","F","G"],"inversions":[0]},
  {"id":"pat.block.I-IV-V-I.C","gen":"AccompanimentPattern","pattern":"blockQuarters","prog":"I-IV-V-
I","key":"C","targetBPM":70}]},
 {"id":"L2","title":"Inversions & Alberti","nodes":[
  {"id":"chord.inv.C","gen":"TriadInversionDrill","key":"C"},
  {"id":"pat.alberti.I-V-vi-IV.C","gen":"AccompanimentPattern","pattern":"alberti","prog":"I-V-vi-
IV","key":"C","targetBPM":72},
  {"id":"tech.hanon1.C","gen":"HanonFigure","n":1,"key":"C","targetBPM":60}]},
 {"id":"L3","title":"Scales: Sharps Family","nodes":["scale.major.G", "scale.major.D", "..."]},
 {"id":"L4","title":"Minor Keys & 7th Chords"},
 {"id":"L5","title":"ii–V–I, Blues & Shell Voicings"},
 {"id":"L6","title":"All 12 Keys, Comping from Lead Sheets"}]}
```

Fill in all levels completely with real nodes (aim for ~250 nodes). Generated keys follow the circle of fifths.

9. Phases

- **MVP (Phase 1):** MusicCore, ExerciseGen 1–11, 13–14 (block, broken, Alberti, root-5-8, ballad), MIDI input (USB + BLE), sampler, metronome, Practice Stage (Keys view plus basic Verovio staff), ScoringKit (pitch, rhythm, evenness, articulation, dynamics, pedal), tempo ladder, daily plan, chord recall FSRS, Reverse ID, Progress charts, Tab parser + fretboard view, SwiftData persistence. Offline. No paywall.
- **Phase 2:** mic input (mono then Basic Pitch polyphonic), Pattern Mixer with backing grooves, lead-sheet comping, 7th/shell/rootless voicings, stride/boogie, iCloud sync, StoreKit 2, Game Center, MIDI out routing.
- **Phase 3:** Vision hand-pose posture hints (advisory), teacher assignment share links, MusicXML import, Apple Watch practice timer, macOS (Designed for iPad / Catalyst).

10. Testing

- Unit tests (XCTest / Swift Testing): interval/chord/scale spelling in all 12 keys (Eb major has Ab not G#), inversion generation, fingering table validity, transposition round-trips, the tab parser cases above, FSRS intervals, tempo ladder, and plan builder minute allocation.
- **Scoring golden tests:** recorded MIDI event JSON fixtures (perfect, 30 ms late finger 4, wrong note, skipped rest, pedal mush, uneven velocity). Assert the expected dimension scores and diagnosis.
- Input tests: `MockMIDIInput`, and a hot-plug disconnect/reconnect mid-take with no crash. BLE reconnect.
- Latency test harness: measure the MIDI-in → highlight round trip and assert app-side processing < 10 ms on an A15+ device in Instruments, logging per-transport (USB/BLE) totals separately.
- UI tests: onboarding, run a take with the mock MIDI, loop bars, start from bar, change tempo.

- Accessibility audit (`performAccessibilityAudit()` in XCUITest), and Dynamic Type at XXXL.
- Performance: Practice Stage at 120 fps on ProMotion iPad. No main-thread audio work.

11. Acceptance criteria (MVP)

1. Connecting a BLE or USB MIDI keyboard works from onboarding in under 60 s. MIDI-in → on-screen highlight is < 10 ms of app processing on top of transport latency (USB about 3–4 ms, BLE about 11–23 ms). Timing scores use packet timestamps, with per-transport calibration verified by a test fixture.
2. Any technique, scale, or chord exercise can be generated in all 12 keys with correct spelling and verified fingerings, rendered on staff and keys.
3. A take produces six dimension scores, a per-finger heatmap, and at least one specific diagnosis tip when a fixture with a late 4th finger is replayed.
4. Wrong notes, skipped rests, and pedal held through chord changes are all penalized (verified by fixtures).
5. Mastery requires passing in Test mode on 2 separate days, and the tempo ladder adjusts automatically.
6. The Today plan respects the chosen minutes ±10% and interleaves at least 2 skills.
7. Pasting a B-string tab line `B|--6-8-10-8-6--|` yields F4 G4 A4 G4 F4 with a suggested RH fingering.
8. The app is fully usable offline, with no account, and passes the accessibility audit with zero critical issues.
9. No copyrighted song arrangements are bundled. Sound-sample licenses are attributed in About.
10. Paywall and billing pass an ethics checklist: no card for the trial, one-tap cancel deep link, a renewal reminder notification, and zero upsell interruptions inside a practice take.
11. Technique sessions enforce a rest prompt after at most 10 minutes of continuous technique and never offer an endurance challenge.
12. README documents the build, architecture, and how to add a generator or curriculum node.

12. Deliverables

The Xcode project (or `Package.swift` workspace) that builds without warnings. `README.md`, `ARCHITECTURE.md`, `TODO.md`. Curriculum JSON, fingerings JSON, and diagnosis rules JSON. Test fixtures. A screenshot set for iPhone 15 Pro and iPad Pro 13".

13. Competitive context for the agent (do not copy, learn from)

- **Simply Piano, flowkey, Skoove, Yousician:** polished onboarding and Wait Mode. Avoid their lenient grading, falling-notes dependency, and billing patterns.
- **Piano Marvel:** sight-reading assessment and real repertoire. Avoid its dated UI and iOS MIDI regressions.
- **Playground Sessions:** bootcamp sequencing. Avoid headphone latency and the early difficulty ceiling.
- **Hanon Pro:** native SwiftUI, Swift Charts trend tracking, and MIDI scoring of accuracy, tempo, and dynamics. Add start-from-bar, MIDI-out playback, and stability.
- **Piano Companion, Tenuto:** chord and scale depth, customization, and pay-once loyalty.
- **The Ultimate Piano (web):** per-note timing and velocity consistency in scales, and inversion accuracy summaries. Match it and exceed it natively.
- **Playy, Chordwise:** custom drill builders (key, mode, chord type, inversion, tempo, rhythm), with no account and no ads.
- **Melodics:** tight timing grading and groove content.